

# Agentic AI Driven Workflow Platform for Semantic Search and Multi Document Reasoning

Ayoub Amiria\*, Ilef Bouallegue<sup>†</sup>, Mazen Tmani<sup>§</sup>, Oussama Guerrish<sup>‡</sup>

\*Paris Dauphine University, Tunisia

<sup>†</sup>National School of Computer Science, Tunisia

<sup>‡</sup>Faculty of Sciences of Tunis, Tunisia

<sup>§</sup>Independent, Tunisia

*Supervised by: Yousri Kessentini*

**Abstract**—Document intelligence systems must process heterogeneous inputs whose format, quality, and structure often require different processing strategies. Fixed pipelines may therefore execute unnecessary or unsuitable operations, increasing computational cost and propagating errors across downstream stages. This paper presents FlowDocs, a modular workflow framework in which document-processing capabilities are represented as typed, composable nodes and organized into graph-based execution workflows. The framework supports preprocessing, optical character recognition (OCR), parsing, retrieval-augmented generation (RAG), clustering, and specialized reasoning components, while enabling content-aware routing between alternative processing paths.

The framework is evaluated through three representative tasks: OCR, document question answering, and topic clustering. Preliminary OCR experiments on synthetically degraded samples show strong sensitivity to rotation, noise, and blur, highlighting the importance of selective preprocessing. On ten DocVQA documents, the fixed RAG configuration achieved 60% exact match, compared with 40% for a larger static pipeline, while rule-based routing successfully selected a parser for native DOCX input and skipped unnecessary OCR. For topic clustering, an LSA-K-Means configuration achieved the best quality-efficiency trade-off among the evaluated alternatives. Overall, the results indicate that additional processing stages do not systematically improve performance and support modular, input-dependent workflow composition. Agentic Planner and Router orchestration is retained as future work.

**Index Terms**—document intelligence, modular workflows, OCR, document question answering, retrieval-augmented generation, content-aware routing, topic clustering, multi-document reasoning

## I. INTRODUCTION

The rapid growth of digital information has fundamentally transformed the scale and complexity of knowledge management. Organizations and digital platforms continuously generate large volumes of documents, ranging from reports, contracts, and scientific publications to forms, spreadsheets, scanned archives, and images. Unlike structured data, however, documents rarely conform to a uniform representation: textual content may coexist with tables, visual elements, handwriting, or complex layouts, while quality, format, and language can vary substantially. This diversity makes documents a particularly rich, but difficult, source of computationally exploitable knowledge.

Extracting value from such information often requires more than a single processing operation. Depending on the characteristics of the input, a document may need to be enhanced, parsed, recognized, normalized, indexed, retrieved, or reasoned over before its content can be reliably exploited. Advances in multimodal document understanding have significantly improved the joint processing of textual, visual, and structural information [8], while retrieval-augmented generation has extended the ability of generative models to exploit external knowledge sources [9]. Agentic approaches have further demonstrated that reasoning models can interact with external tools and dynamically select actions during task execution [10].

Yet progress in individual capabilities does not by itself determine how they should be combined. A degraded scanned document may benefit from image enhancement and optical character recognition, whereas a digitally generated document may be parsed directly; OCR-based processing itself can also introduce computational overhead and propagate recognition errors to subsequent stages [11]. Similarly, retrieval, comparison, clustering, and reasoning tasks may require substantially different processing paths. In practice, these capabilities are frequently integrated through predefined or manually engineered pipelines, making adaptation to new document characteristics or task objectives increasingly difficult as the number of available processing components grows. Executing unnecessary stages may also increase computational cost and latency without providing a proportional improvement in result quality.

This exposes a broader challenge in document intelligence: the problem is not only how well individual capabilities perform, but how the appropriate capabilities should be selected, ordered, and coordinated for a given document and objective. The convergence of document understanding, retrieval-augmented generation, multimodal processing, and tool-using reasoning now provides the foundations for approaching document processing as a dynamic composition problem rather than a rigid chain of predefined operations. Such a perspective motivates the investigation of modular and adaptive workflows in which processing and reasoning capabilities are composed according to the content and context of the task, while explicitly considering the trade-off between result quality,

computational cost, and execution latency.

## II. RELATED WORK

Recent document intelligence research has increasingly moved from text-only processing toward multimodal representations that jointly exploit textual, visual, and structural information. LayoutLMv3 [8], for instance, introduces a unified Transformer architecture based on masked text and image modeling together with word-patch alignment. By combining OCR-derived textual tokens and their spatial positions with visual document representations, it supports both text-centric tasks, such as form understanding and document question answering, and image-centric tasks such as document classification and layout analysis. This line of work demonstrates the importance of preserving document structure rather than reducing documents to plain text alone.

However, OCR-dependent architectures introduce an additional processing stage whose errors and computational cost can affect downstream tasks. Donut [11] addresses this limitation through an end-to-end OCR-free Document Understanding Transformer that directly maps document images to structured outputs. The work highlights three limitations of conventional OCR-based pipelines: computational overhead, limited adaptability across languages and document domains, and the propagation of recognition errors to subsequent processing stages. These observations are particularly relevant to modular document workflows, where the usefulness of OCR and preprocessing operations may depend strongly on the characteristics of the input document.

More recently, document understanding has been extended toward retrieval-augmented and agentic processing of long, visually rich documents. VLD-RAG [13] combines page-level visual representations with structured textual information in a multimodal index and performs hybrid sparse and dense retrieval. Its Retrieval, Answer, and Validation agents iteratively refine evidence selection when the retrieved context is insufficient, supporting cross-page reasoning over complex documents. This illustrates a broader transition from isolated document models toward the coordination of multiple processing and reasoning capabilities.

These approaches primarily improve individual document representations, end-to-end understanding models, or task-specific retrieval architectures. Our work is complementary and focuses instead on the *workflow level*: heterogeneous capabilities such as preprocessing, OCR, parsing, retrieval, reasoning, and clustering are exposed as modular typed nodes that can be composed into different execution paths. Rather than assuming a single processing strategy for all inputs, the evaluated system compares fixed, static, and deterministic rule-based workflows to study when individual operations should be applied or skipped. Agentic planning and routing extend this architecture conceptually, while their experimental evaluation is left for future work.

## III. PROBLEM FORMULATION

Let  $D$  denote a heterogeneous input document (or document set), composed of elementary units  $d_1, \dots, d_m$  (pages, tables, figures, form fields, text blocks), each with an a priori unknown content type  $t_i$  (digital text, scanned text, table, figure, handwriting, etc.). Let  $q$  denote the user objective (extraction, QA, comparison, clustering, or multi-document reasoning).

### A. Typed Node Interfaces

FlowDocs defines a set of nodes  $\mathcal{N} = \{n_1, \dots, n_K\}$ , organized into functional families (Section IV): ingestion & preprocessing, content extraction, structural analysis & normalization, indexing & RAG, and specialized reasoning agents. Each node declares a typed interface

$$n_k : \mathcal{X}_k \rightarrow \mathcal{Y}_k, \quad (1)$$

and two nodes  $n_k, n_{k'}$  can be chained only if  $\mathcal{Y}_k$  is type-compatible with  $\mathcal{X}_{k'}$ . Beyond enabling composability, this interface acts as a contract: each node additionally emits a confidence or quality signal  $c_k \in [0, 1]$  alongside  $\mathcal{Y}_k$ , allowing downstream nodes and the orchestrator to reason about the reliability of an upstream output before consuming it.

### B. Error Propagation in Node Chains

For a chain  $\sigma = (n_{i_1}, \dots, n_{i_L})$  applied to  $D$ , each node's output depends on its predecessor's output:

$$a_j = n_{i_j}(D, q, a_{j-1}), \quad a_0 = \emptyset. \quad (2)$$

An error  $\epsilon_j$  introduced at step  $j$  (e.g., a misread character, a mistyped table cell, a wrong language detection) can propagate and compound through subsequent nodes that consume  $a_j$  as input, so that the final error is a function of the entire path:

$$\epsilon_{\text{final}} = f(\epsilon_1, \epsilon_2, \dots, \epsilon_L \mid \sigma). \quad (3)$$

A fixed, content-agnostic pipeline forces every document through the same  $\sigma_{\text{fixed}}$ , regardless of  $D$ 's actual type distribution  $\{t_i\}$ . This has two consequences: (i) nodes unsuited to  $D$  are invoked unnecessarily, each acting as an additional potential error source; (ii) nodes better suited to a given  $t_i$ , which could have limited  $\epsilon_j$ , are never selected. Interoperability alone (respecting  $\mathcal{X}_k/\mathcal{Y}_k$  types) is therefore necessary but not sufficient to control  $\epsilon_{\text{final}}$ .

### C. Dynamic Orchestration as Error Control

FlowDocs treats node selection and ordering as decisions that actively shape the propagation path, not just a scheduling problem. For each  $(D, q)$ , an execution graph  $G(D, q) = (\mathcal{N}_{\text{active}}, \sigma)$  is constructed via three coordinated components:

- **Planner Agent:** decomposes  $q$  into a task plan  $P$  given  $D$ 's context (the *what*);
- **Router Agent:** given  $P$  and the detected content types in  $D$ , selects the node(s)  $\mathcal{N}_{\text{active}} \subseteq \mathcal{N}$  best matched to each subtask, using node interfaces and, where available, upstream confidence  $c_k$  (the *who*);
- **Workflow Orchestrator:** fixes a type-valid execution order  $\sigma$  over  $\mathcal{N}_{\text{active}}$ , propagating  $a_j$  and  $c_j$  between nodes

and enabling re-routing when an intermediate  $c_j$  falls below a reliability threshold (the *how*).

#### D. Optimization Objective

The system must select  $G(D, q)$  to jointly limit error propagation, maximize task performance, and bound computational cost:

$$\max_{G(D, q)} \text{Quality}(a, D, q) - \alpha \cdot \epsilon_{\text{final}}(G(D, q)) - \lambda \cdot \text{Cost}(G(D, q)), \quad (4)$$

where  $\text{Quality}(\cdot)$  is task-dependent (accuracy, F1, retrieval or clustering metrics, per Section V),  $\text{Cost}(\cdot)$  aggregates nodes executed, LLM calls, tokens, and latency, and  $\alpha, \lambda$  weight error sensitivity against efficiency. This operationalizes our research question: whether typed inter-node interfaces, combined with dynamic orchestration, can govern information flow so as to mitigate error propagation and optimize task performance across heterogeneous document workflows, compared to fixed or rule-based execution.

### IV. PROPOSED APPROACH: FLOWDOCS

The formulation in Section III casts document processing as a problem of typed node composition and dynamic orchestration under an error-and-cost objective. We now instantiate this formulation as FlowDocs, a graph-based execution platform in which the abstract components introduced above—typed nodes  $\mathcal{N}$ , the execution graph  $G(D, q)$ , and the Planner/Router/Orchestrator triad—are realized as a concrete, extensible system.

#### A. Overall Architecture

FlowDocs is a graph-based execution platform for agentic document processing. Users compose a directed acyclic graph  $G = (\mathcal{N}_{\text{active}}, \sigma)$  from a library of typed nodes (Section III); a backend execution engine resolves the graph, invokes each node’s `execute` method in topological order, and propagates outputs  $a_j$  and confidence signals  $c_j$  along edges. Nodes are self-contained, independently testable units, each pairing a thin orchestration wrapper with an underlying service implementation, allowing individual capabilities to be upgraded without altering the surrounding graph. Figure 1 illustrates the overall architecture.

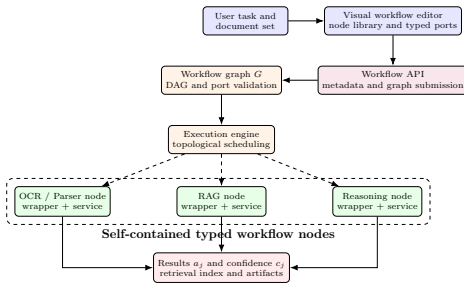


Fig. 1. FlowDocs architecture: visual workflow composition, typed graph validation, topological execution, modular processing nodes, and propagated results.

#### B. Node Families

Nodes are organized into six functional families, following the taxonomy introduced in Section III. Figure ?? summarizes each family, its role in the pipeline, and representative nodes; full per-node specifications (role, input/output types, owner) are provided in Appendix ??.

#### C. Typed Input/Output Propagation

Each node exposes typed ports  $n_k : \mathcal{X}_k \rightarrow \mathcal{Y}_k$ , as formalized in Section III, and the execution engine validates port-type compatibility at edge-creation time, rejecting incompatible connections. This typed contract is what lets an error introduced at one node be traced and contained: since the Router only connects a node to inputs it can legitimately consume, a low-confidence output  $c_j$  can be intercepted or re-routed before it propagates further downstream, rather than being silently passed forward.

#### D. Planning, Routing and Orchestration

At runtime, the Planner decomposes the user objective  $q$  into a task plan  $P$  without committing to specific nodes. The Router, an LLM-backed component, is prompted with  $P$ , the detected document structure, and a fixed catalogue of available node capabilities, and returns a structured, ordered list of (node, reason) pairs selecting  $\mathcal{N}_{\text{active}}$ . Unlike a closed, binary multi-label agent dock [12], our Router performs constrained generation over an open node catalogue, allowing it to justify each selection and to be extended with new node types without retraining. The Orchestrator then fixes a type-valid execution order  $\sigma$  over  $\mathcal{N}_{\text{active}}$ , as described in Section IV.

#### E. Specialized Agents

Beyond extraction, several agents reason *over* already-extracted or retrieved content: Table, Text Reasoning, Figure/Image, Comparison, Summarization, Translation, Topic Clustering, and Sentiment agents each consume a narrow, well-typed input and follow the same LLM-prompted specialization pattern. The **Answer Generator** synthesizes their outputs into a single response with an associated confidence label, and the **Validation/Critic Agent** independently cross-checks this response against the retrieved evidence — implemented as a single verification pass rather than a multi-turn adversarial protocol, favoring lower latency over exhaustive reliability checking.

### V. EXPERIMENTAL METHODOLOGY

#### A. Datasets

The evaluation should be conducted on a heterogeneous document collection reflecting the diversity of inputs encountered in document-intelligence workflows.

TABLE I  
DATASETS USED IN THE EXPERIMENTAL EVALUATION

| Dataset | Documents             | Format                | Characteristics                                                          | Tasks                 |
|---------|-----------------------|-----------------------|--------------------------------------------------------------------------|-----------------------|
| D1      | DocVQA                | Multi-document        | Related documents                                                        | Comparison, reasoning |
| D2      | 20 Newsgroups         | Mixed                 | 18,846 documents distributed across 20 topics                            | Clustering            |
| D3      | Synthetic OCR dataset | Synthetic text images | Clean, low contrast, +3° rotation with Gaussian noise, and Gaussian blur | OCR sanity check      |

### B. Experimental Tasks

The experimental evaluation covers three representative document-intelligence tasks selected to exercise different parts of the processing architecture, from low-level extraction to multi-document reasoning.

- **Optical Character Recognition (OCR):** Evaluation of text recovery from scanned or image-based documents, including degraded inputs for which preprocessing may be required.
- **Document Question Answering:** Evaluation of the ability to retrieve and reason over document content in response to natural-language questions.
- **Topic Clustering:** Evaluation of the ability to organize document collections into semantically coherent groups without predefined labels.

### C. Document QA / RAG Configurations

The document QA experiments rely on a retrieval-augmented generation workflow organized according to an “index once, query many” principle: documents are processed and indexed once, whereas user questions are handled through a separate query path. Figure 2 shows the core RAG pipeline used as the reference configuration.

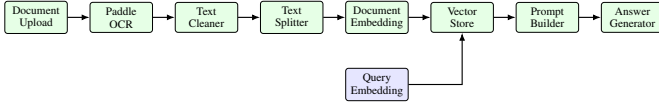


Fig. 2. Core RAG pipeline used in the document question-answering experiments.

**R-Fixed** corresponds to the pipeline shown in Fig. 2, including document upload, OCR, text cleaning, text splitting, document embedding, vector storage, query embedding, prompt construction, and answer generation. **R-Static** extends this workflow with denoising, contrast enhancement, text correction, critic validation, and feedback collection. **R-Rule** introduces deterministic routing: image files are sent to OCR, whereas DOCX and PDF documents are sent to the document parser, with the unused branch skipped by the conditional executor.

#### • Task and Configuration — OCR

We evaluate the OCR Node (Tesseract-based, as implemented in Section IV) across the four conditions above, to observe sensitivity to distinct types of visual degradation that preprocessing nodes (denoising, deskewing) are intended to correct.

TABLE II  
OCR PERFORMANCE ACROSS SYNTHETIC DEGRADATION TYPES ( $N = 6$ )

| Condition        | CER ↓ | WER ↓ |
|------------------|-------|-------|
| Clean            | 0.023 | 0.292 |
| Low contrast     | 0.020 | 0.262 |
| Rotation + noise | 0.169 | 0.651 |
| Gaussian blur    | 1.000 | 1.000 |

### D. Baselines and Proposed Configuration

#### • Topic Clustering

Two representative baselines were selected to compare a classical lexical approach, a latent-semantic representation, and the current project implementation.<sup>1</sup>

TABLE III  
EVALUATED CLUSTERING CONFIGURATIONS

| ID | Representation                            | Clustering                              |
|----|-------------------------------------------|-----------------------------------------|
| B1 | Word TF-IDF (1–2 grams, 16k features)     | K-Means, $k = 20$                       |
| B2 | TF-IDF + 100-dim. LSA                     | K-Means, $k = 20$ [14], [15]            |
| B3 | Word + char. TF-IDF (3–5 grams, wt. 0.35) | MiniBatchKMeans, $k = 20$ , batch = 256 |

Each configuration was executed five times using random seeds 42–46, while all remaining parameters were kept fixed.

### E. Ablation Study

For RAG, the ablation compares R-Fixed with R-Fixed plus denoising and contrast enhancement, and with the complete R-Static configuration. A second qualitative test applies salt-and-pepper noise and contrast reduction to an address document to determine whether preprocessing is beneficial only when the input is degraded.

### F. Evaluation Metrics

Consistent with the system-wide metrics defined in Section IV, we report, for the tasks above:

#### • Evaluation Metrics — OCR

We report CER and WER between the OCR Node’s output and ground truth. Tesseract was robust to moderate contrast reduction but failed completely under Gaussian blur, producing no correctly recognized words. Rotation combined with noise degraded CER by roughly  $7\times$  and WER by roughly  $2\times$  relative to clean input. These results are preliminary and based on a small synthetic set; evaluation on the full D2 dataset is left for future work, and blur correction is noted as a gap not currently covered by the implemented preprocessing nodes.

- **Topic Clustering:** Clustering performance was evaluated in terms of both partition quality and computational efficiency.<sup>2</sup>Reference topic labels were used only for post-hoc evaluation and were never provided during clustering.

TABLE IV  
EVALUATION CRITERIA

| Type       | Metric         | Purpose                                  |
|------------|----------------|------------------------------------------|
| External   | ARI ↑          | Agreement with reference topics          |
| External   | NMI ↑          | Shared information with reference topics |
| Internal   | Silhouette ↑   | Cluster cohesion and separation          |
| Efficiency | Runtime ↓      | End-to-end execution time                |
| Efficiency | $\Delta$ RSS ↓ | Additional memory consumption            |

<sup>1</sup>TF-IDF: Term Frequency–Inverse Document Frequency; LSA: Latent Semantic Analysis.

For comparability, the Silhouette Score was computed using cosine distance in a common word-level TF-IDF space on a fixed sample of 2,000 documents.

## VI. RESULTS AND DISCUSSION

### A. Document QA / RAG Results

Table V reports the verified RAG results on ten DocVQA documents. R-Fixed achieved the strongest exact-match result in this run. R-Static was slower and less accurate despite using additional preprocessing and validation stages. R-Rule produced a similar image-only result because all ten inputs were images; its main contribution is support for mixed file formats.

TABLE V  
VERIFIED RAG RESULTS ON TEN DOCVQA DOCUMENTS

| Config.  | Nodes  | Exact | Cont. | Calls | Time |
|----------|--------|-------|-------|-------|------|
| R-Fixed  | 9      | 60%   | 60%   | 1     | 51 s |
| R-Static | 14     | 40%   | 50%   | 2     | 73 s |
| R-Rule   | Routed | 50%   | 50%   | 1     | —    |

For the DOCX routing demonstration, R-Rule selected the parser branch, skipped OCR, extracted the contract text, and answered “15,000 EUR” in 4.3 seconds. This demonstrates format coverage that the fixed OCR-only configuration cannot provide.

### B. RAG Preprocessing Ablation

Table VI isolates the effect of image preprocessing. Adding denoising and contrast enhancement to R-Fixed reduced exact match from 60% to 30%. The complete static configuration recovered part of this loss but remained below the base configuration.

TABLE VI  
RAG PREPROCESSING ABLATION ON TEN DOCVQA DOCUMENTS

| Configuration           | Nodes | Exact | Contains |
|-------------------------|-------|-------|----------|
| R-Fixed                 | 9     | 60%   | 60%      |
| R-Fixed + preprocessing | 11    | 30%   | 50%      |
| R-Static                | 14    | 40%   | 50%      |

On a clean address document, R-Fixed recovered “WASHINGTON, D. C. 20036”, whereas preprocessing produced “WASH NGTON. D C 20038”. Conversely, on a synthesized noisy and low-contrast version, plain OCR recovered only a small part of the address, while denoising and contrast enhancement recovered most of its structure, although character errors remained. Preprocessing is therefore input-dependent and should not be applied unconditionally.

<sup>2</sup>ARI: Adjusted Rand Index; NMI: Normalized Mutual Information; RSS: Resident Set Size.

### C. Topic Clustering Results

Table VII reports the results over five runs. B2 (LSA + K-Means) achieved the best clustering quality while maintaining a runtime comparable to B1.

TABLE VII  
CLUSTERING RESULTS OVER FIVE RUNS

| Method    | ARI $\uparrow$                      | NMI $\uparrow$                      | Silhouette $\uparrow$               | Runtime (s) $\downarrow$ | $\Delta$ RSS (MB) $\downarrow$ |
|-----------|-------------------------------------|-------------------------------------|-------------------------------------|--------------------------|--------------------------------|
| B1        | 0.101 $\pm$ 0.011                   | 0.261 $\pm$ 0.028                   | 0.001 $\pm$ 0.002                   | 6.42 $\pm$ 0.13          | 242                            |
| <b>B2</b> | <b>0.132 <math>\pm</math> 0.002</b> | <b>0.291 <math>\pm</math> 0.007</b> | <b>0.005 <math>\pm</math> 0.002</b> | 6.60 $\pm$ 0.08          | 258                            |
| B4        | 0.019 $\pm$ 0.007                   | 0.081 $\pm$ 0.017                   | -0.020 $\pm$ 0.007                  | 24.52 $\pm$ 0.19         | 689                            |

$\uparrow$  higher is better;  $\downarrow$  lower is better.

Figure 3 confirms the advantage of the latent semantic representation used by B2. In contrast, B4 shows lower external agreement and a negative Silhouette Score, indicating weaker cluster separation.

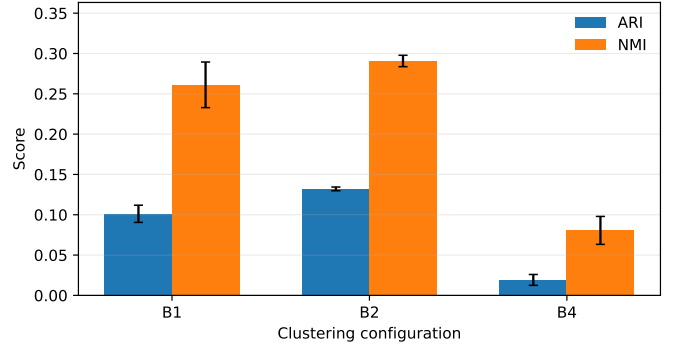


Fig. 3. External clustering quality across the evaluated configurations.

B3 also required substantially more computational resources, with approximately  $3.7\times$  the runtime and  $2.7\times$  the additional memory of B2. The quality–efficiency trade-off is illustrated in Fig. 4.

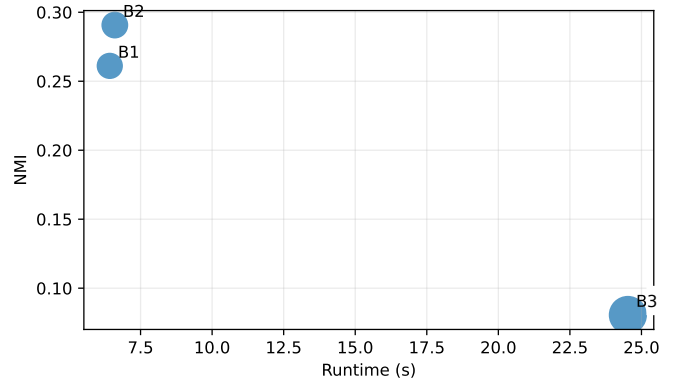


Fig. 4. Clustering quality–efficiency trade-off.

Overall, LSA followed by K-Means provided the most favorable balance between clustering quality and computational cost on this dataset. These results also suggest that a single clustering configuration may not be optimal for all corpora, supporting the use of adaptive strategy selection according to document characteristics and scale.

## VII. FUTURE WORK

The Future work will implement the agentic Planner–Router–Orchestrator configuration described in Section IV-D. The Planner will decompose the user objective, the Router will select typed nodes from the catalogue, and the Orchestrator will execute only the selected subgraph.

## VIII. CONCLUSION

This paper presented FlowDocs, a typed modular platform for composing OCR, RAG, and topic-clustering workflows over heterogeneous documents. The verified RAG evaluation showed that the minimal fixed configuration achieved 60% exact match, while the larger static configuration achieved 40%. Rule-based routing selected a parser for DOCX input, skipped OCR, and answered a contract question in 4.3 seconds. The preprocessing ablation showed that image transformations can recover more structure from degraded scans while corrupting clean text when applied unconditionally. These results support modular and deterministic workflow composition; the agentic routing configuration remains future work.

## IX. CONTRIBUTIONS

- **Ayoub Amiria** : developed and integrated the Contrast Enhancer, Document Parser, Date Normalizer, Masker, Vector Database, Comparison Agent, and Topic Clustering nodes, and built the frontend workflow editor integrating these components. In the paper, he defined the overall structure and authored the Abstract, Introduction, and Related Work. He also contributed to the methodology by defining the organization of the clustering experiments.
- **Ilef Bouallegue** : designed and implemented nine backend nodes *Ingestion & Preprocessing* (File Converter, Watermark Remover), *Content Extraction* (OCR Node, Table Extractor), *Indexing & RAG* (Embedding Node), and *Orchestration & Reasoning / Specialized Agents* (Router Agent, Table Agent, Translator, Answer Generator Agent). also authored Section III (Problem Formulation) and contributed the OCR evaluation (Table II). Last but not least, the conclusion.
- **Mazen Temani** : implemented the backend node SDK and registry and the document-processing and reasoning nodes (document upload, handwriting OCR, regex and semantic extraction, structure analysis, gap checking, prompt building, feedback collection, sentiment analysis, and critic validation). He built the end-to-end RAG/QA workflow with a persistent index and QA Chat, and developed the rule-based router and conditional executor (R-Rule/B3). He contributed to the frontend visual workflow editor. In the paper, he authored the Document QA/RAG methodology, results, and preprocessing ablation, and produced the baseline figures.
- **Oussama Guerrish** : initialized the project structure and implemented the image denoising node, the language and orientation detector, the summarizer, and the bar/QR code extraction node.

## REFERENCES

- [1] G. Eason, B. Noble, and I. N. Sneddon, “On certain integrals of Lipschitz-Hankel type involving products of Bessel functions,” *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, April 1955.
- [2] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.
- [3] I. S. Jacobs and C. P. Bean, “Fine particles, thin films and exchange anisotropy,” in *Magnetism*, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.
- [4] K. Elissa, “Title of paper if known,” unpublished.
- [5] R. Nicole, “Title of paper with only first word capitalized,” *J. Name Stand. Abbrev.*, in press.
- [6] Y. Yorozu, M. Hirano, K. Oka, and Y. Tagawa, “Electron spectroscopy studies on magneto-optical media and plastic substrate interface,” *IEEE Transl. J. Magn. Japan*, vol. 2, pp. 740–741, August 1987 [Digests 9th Annual Conf. Magnetism Japan, p. 301, 1982].
- [7] M. Young, *The Technical Writer’s Handbook*. Mill Valley, CA: University Science, 1989.
- [8] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, “LayoutLMv3: Pre-training for document AI with unified text and image masking,” in *Proc. 30th ACM Int. Conf. Multimedia (MM)*, 2022, pp. 4083–4091.
- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [11] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, “OCR-free document understanding transformer,” in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 498–517.
- [12] A. Lassoued, M. A. Souibgui, and Y. Kessentini, “ORCA: Orchestrated reasoning with collaborative agents for document visual question answering,” *arXiv preprint arXiv:2603.02438*, 2026.
- [13] S. Kim, “VLD-RAG: Agentic vision-language retrieval-augmented generation for long, visually-rich multi-page documents,” *arXiv preprint arXiv:2607.24748*, 2026.
- [14] Scikit-learn developers, “Clustering text documents using k-means,” *Scikit-learn Documentation*.
- [15] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, “Indexing by latent semantic analysis,” *Journal of the American Society for Information Science*, vol. 41, no. 6, pp. 391–407, 1990.